Decentralized Machine Learning: Optimal Topology for a Given Data Distribution

Berkay Guler and Ryuto Kitagawa

Problem Statement

Decentralized Learning (DL) aims to train a model through parameter sharing between nodes without actual data exchange. In this setting, each node has only access to its local data. We consider a neural network and the decentralized stochastic gradient descent (DSGD) algorithm to learn the digit recognition task on MNIST dataset. At each round, once all nodes have completed an update step on their model, each node calculates the weighted average of its model and the neighbouring nodes' models to obtain a new local model for the next round. These rounds are repeated as many times as necessary until some stopping criteria is met. The network topology plays a crucial role in the performance of the DL. In this work, we address the problem of learning an optimum topology for DL.

Optimization Equation

$$\max_{G} \frac{1}{n}\sum_{i=1}^{n} P_G(h_{\theta_i^*}, \mathcal{D}_i)$$

$$\text{subject to } \max_{i\in V}(\deg(i)) \leq k$$

Variables such as the number of communication rounds and batch size are key factors which influence the performance of the DSGD algorithm. However, they are fixed and not considered part of the constraints in the equation. 3 optimization algorithms are tested on this project. Two are black box algorithms from class, and one is a composite optimization algorithm.

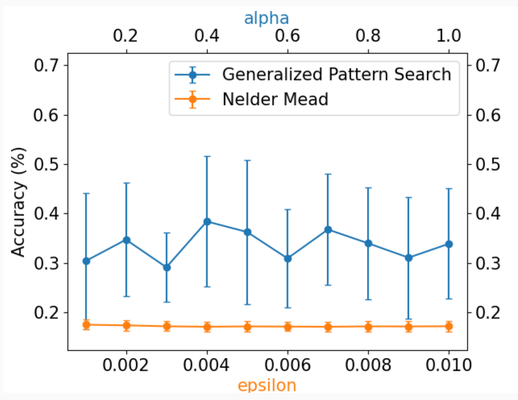
- Generalized Pattern Search
- Nelder Mead
- Composite Algorithm

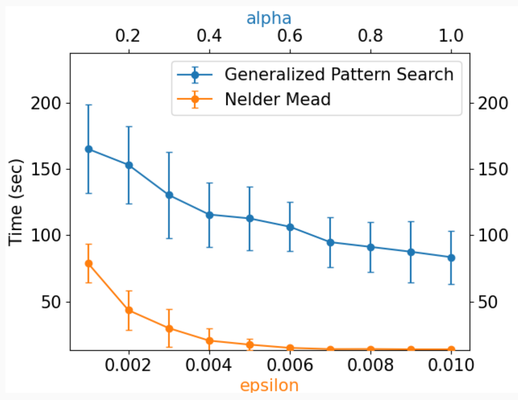
Decentralized Learning Algorithm

Algorithm 1: Decentralized Stochastic Gradient Descent

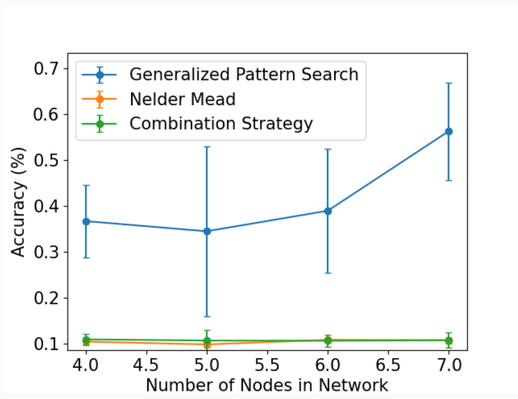
Data: Initial parameters $\theta_i(0)$ for each node i, step size α, maximum iterations T, and weight matrix W
Result: Optimized parameters θ_i for each node i
for $t = 0$ **to** $T-1$ **do**
  for *each node i in parallel* **do**
    Sample $Z_i = \{(x_j^{(i)}, y_j^{(i)}) | j = 1, 2, ..., N\}$ from $\mathcal{D}_i$;
    Compute stochastic gradient: $\nabla_{\theta_i} L(Z_i, h_{\theta_i})$;
    Collect weights from neighbours: $\theta_i(t+1/2) = \sum_{j\in\mathcal{N}_i} W_{ij}\theta_j(t)$;
    Update weights with gradient:
    $\theta_i(t+1) \leftarrow \theta_i(t+1/2) - \alpha\nabla_{\theta_i} L(Z_i, h_{\theta_i})$;

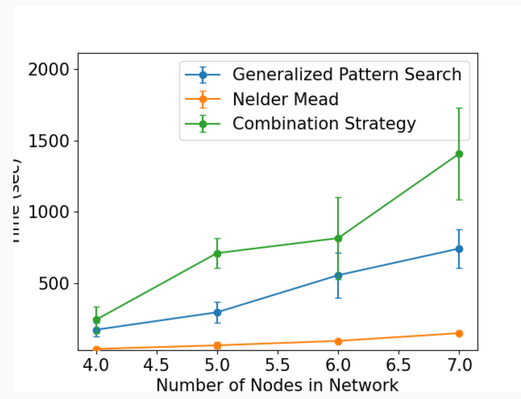
Optimizing on 3 Fully Connected Nodes, with 10 Rounds of Communication





Optimizing on a Restricted Degree Network of Nodes, with 10 Rounds of Communication





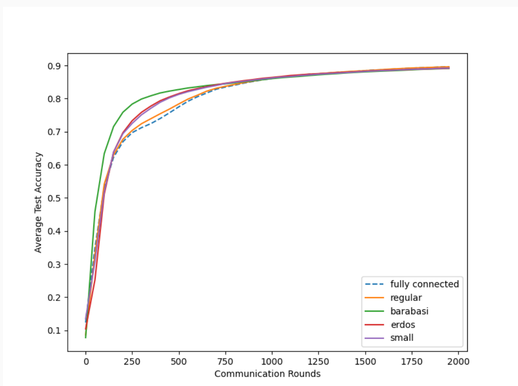
Analysis of Blackbox Output Performance

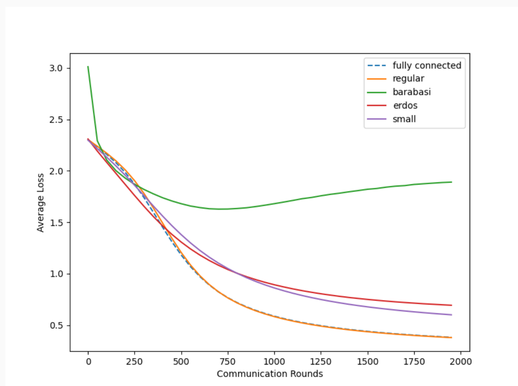
In the figure above, we observe on average the Generalized Pattern Search method outperforms the Nelder Mead algorithm for the accuracy of the weight matrix. However, due to high variance and a limited number of trials, it is challenging to tell if the performance improves as α increases. Though performance does seem consistent across different values of α. Interestingly, the Nelder Mead algorithm maintains an average performance of 0.175 across varying ϵ values. As a baseline, we compared this to the weight matrix with the best performance across 9 random adjacency matrices, where the Nelder Mead algorithm performed slightly better with the random method obtaining an average of 0.171. However, the limited number of trials constrains the robustness of this conclusion.

Analysis of Blackbox Time Performance

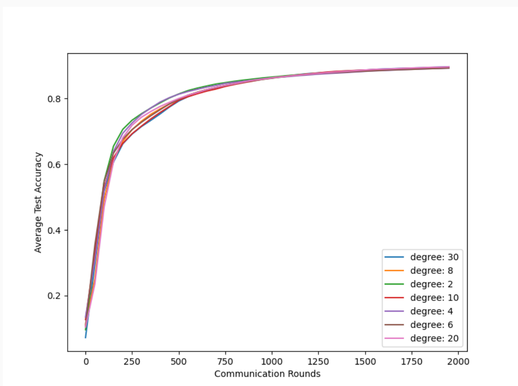
The Generalized Pattern Search algorithm, despite its significantly superior output, does also take considerably longer than the Nelder Mead method. Intriguingly, for the smallest values of α and ϵ, the Nelder Mead algorithm makes more function calls (graph not shown) than the Generalized Pattern Search algorithm but still operates faster. The reason for this is unclear, though some background python optimization is suspected to be the culprit. Both algorithms show decreased wall time with increasing parameter values. These results have the least variance, lending them higher reliability, though the limited trials still warrants a cautious interpretation of these results.

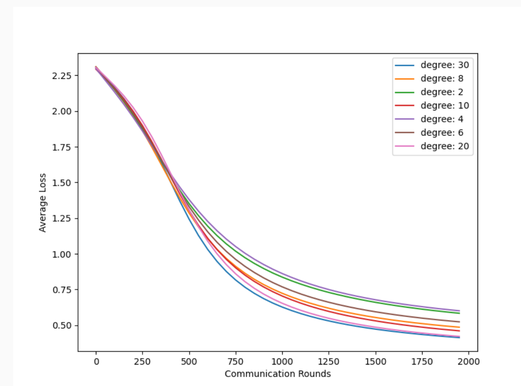
Effect of Topology for Larger Scale DL





Effect of Connectivity for Larger Scale DL





Composite Algorithm Pseudocode

Algorithm 2: Composite Optimization Algorithm

Data: Accuracy evaluation function f
Result: Optimized W to maximize f
Run Generalized Pattern Search on a random starting matrix to find the optimal set of edges to connect;
Fine tune what the weights of these edges should be using the Nelder Mead Function;

Large Scale Experiments

We observe that homogeneous topologies such as random regular graph and complete graph performs the best in iid data setting. However, heteregeneous topologies such as Barabasi-Albert and Watts-Strogatz outperform homogenous topologies at earlier communication rounds. We also observe that some sparse topologies such as Watts-Strogatz provides a decent trade-off between communication overhead and convergence rate as their performance does not depreciate greatly with increasing sparsity.

LaTeX TikZposter